# 实验 6 多播和广播程序设计

## 实验目的

1、掌握使用数据报套接字编写通信程序的方法；

2、会接收和发送广播报文；

3、掌握多播通信原理及其应用；

4、掌握网络多播通信程序的设计方法。

## 实验环境

联网并运行 Linux 操作系统的 PC 机

## 实验内容

本程序基于数据报套接字、使用 P2P 模式进行通信。通过在键盘上输入命令来指出采用点到点、广播还是多播来发送消息。对本程序的要求如下：

1、可以发送点到点报文；

2、可以发送广播报文；

3、可以接收点到点报文；

4、可以接收广播报文；

5、可通过指定多播组地址来加入到特定的多播组；

6、可向已加入的多播组发送信息；

7、可接收已加入多播组中的多播信息。

本程序可以接受的命令有：

- bcast：以广播(broadcast)方式发送消息。命令格式： bcast <消息><CR-LF>
- mcast：以多播(multicast)方式发送消息。命令格式：mcast <组地址> <消息><CR-LF>
- ucast：以单播(unicast)方式发送消息。命令格式：ucast <IP 地址> <消息><CR-LF>
- jgrp：加入指定的多播组(join the group)。命令格式：jgrp <组地址><CR-LF>
- dgrp：从多播组中退出(drop from group)。命令格式：dgrp<CR-LF>
- exit：结束程序运行。命令格式：exit<CR-LF>

命令行各部分之间使用空格隔开，以回车符结束。组地址和单播目的地址均使用点分十进制字符串方式输入。命令名不区分大小写字母。

# 实验指导

1、对等模式（P2P 模式）

P2P 通信模式中，通信双方没有服务器和客户机之分，它们的地位对等，通信的每一方都有可能在没有发送数据之前接收到其它主机发送的数据。而通信的每一方也都可以在没有接收到数据时主动向其它主机发送数据。因此，通信的每一方都应该同时具有服务器和客户端的功能。

UDP 通信程序的 P2P 模型如**错误!未找到引用源。**6-1 所示。

由于通信双方发送和接收消息是异步进行的，因此采用 P2P 模型编写程序时，通信双方必须能够同时处理发送和接收消息的工作。为了能够让程序同时进行消息的发送和接收，可以在通信程序设计中使用多线程技术，也可以使用非阻塞编程技术或者异步编程技术来实现，还可以采用 Linux 支持的如 select、epoll 等 I/O 模型来实现。

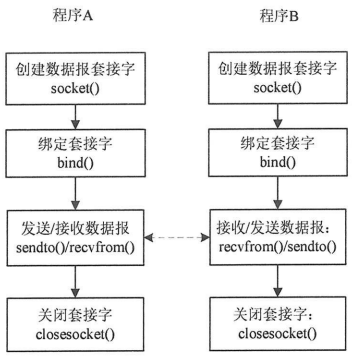



**图 6-1 UDP 通信程序的 P2P 模型**

2、关于广播

广播是指把数据报发送给一个网络中的所有主机。其主要用于：（1）希望在本地网络中寻找一个资源，而又不知道该资源的地址；（2）一些重要的功能，如：路由器要求把它们的信息发送给所有可以找到的相邻机等。

在发送广播报文时，需要使用广播地址。比如：目的地址为 255.255.255.255，则是用于向主机所在网络内的所有主机发送广播报文；主机号全 1 的地址，网络号不是全 1，则是向特定网络中的所有主机广播。不过为了安全起见，现在很多路由器都不再转发广播报文。

虽然利用 TCP 协议与网络中所有主机分别建立 TCP 连接，然后再向每一台主机分别发送报文来实现广播。但是，采取这种方式来实现广播，一方面效率很低，二方面需要占用大量的资源。因此，应用层的广播一般都是通过无连接的 UDP 协议来实现，即：通过使用数据报套接字来实现广播程序。

在使用数据报套接字编程时，可以使用预先定义的宏 INADDR_BROADCAST 作为目的地址来发送广播报文。此外，还需要使用 setsockopt() 函数把此套接字的 SOL_SOCKET 层选项 SO_BROADCAST 设置为 TRUE，以允许在该套接字发送广播报文。若仅仅是想接收广播报文，而不

需要发送广播报文，则不需要设置该选项。

除了要接收直接发送给本机的点到点报文外，还需要接收广播报文，在程序设计中不需要做特殊处理，只要程序绑定到本地地址，比如：INADDR_ANY，并在广播所在 UDP 端口上进行接收即可。如果仅接收广播报文，而不接收点到点报文，那么程序就需要绑定到广播地址 INADDR_BROADCAST 进行接收。

3、关于多播

多播是指把数据报发送给网络中的部分主机。一个进程只有在加入到一个多播组成为该组的成员后，该多播组的数据才会转发给该进程。

组成员身份的概念是 IP 多播的核心，只有多播组的成员才会接收发往该多播组的数据报。多播组通过 IP 多播地址（D 类地址）来标识，其范围是从 224.0.0.0 到 239.255.255.255（在无类域间路由表示法中指定为 224.0.0.0/4）。这些 D 类地址称为组地址。

源主机以 D 类组地址为目标地址把多播数据报发送到网络上；多播路由器则把多播数据报转发到多播组成员所在的网络；主机从网络中接收到多播数据报后，若发现主机中有进程加入到该数据报的目标地址所指出的多播组，则会将该多播报文接收并转交对应的进程；加入到多播组的进程接收到多播报文后，再对接收到的报文进行解析和处理，这样，就完成了多播报文从发送、传输、接收和处理整个过程。

多播报文的传输由多播路由器完成。为了能够让路由器知道其所连接的网络有某个多播组的成员，拟加入到多播组的主机，需要通过发送注册报文来通知本地路由器自己需要加入的多播组，以便路由器将目标为该多播组的数据报转发到该多播组成员所在的网络。

在启用 IP 多播的 Intranet 中，任意主机可以将 IP 多播数据报发送到任意组地址，任意主机可以从任意组地址（与其位置无关）接收 IP 多播数据报。为了实现此功能，Intranet 中的主机和路由器必须支持 IP 多播。主机使用 Internet 组管理协议(IGMP)设置组成员身份。路由器使用多播路由协议转发多播数据。

IGMP 协议即因特网组管理协议（Internet Group Management Protocol）是与多播通信相关的一个必要的但不是充分的协议。IGMP 负责管理多播组成员关系，它把连接在网络上的主机的成员关系状态信息传送给多播路由器。

下图说明一个启用多播的 Intranet。

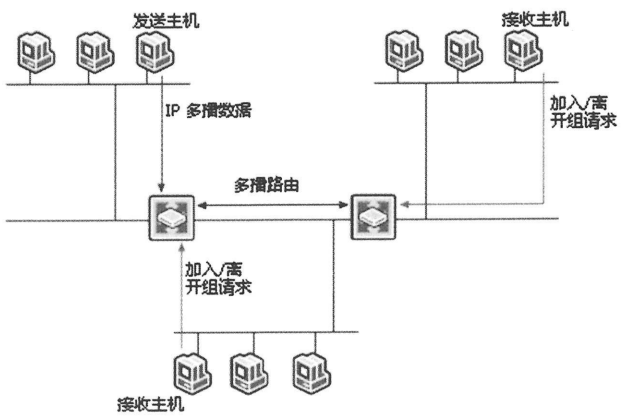



图 6-2 启用多播的 Intranet

在此图中，主机和路由器均已启用多播，所以，可以执行下列操作：

- 主机将多播数据报发送到指定的组地址。
- 路由器将多播数据报转发到任何包含组成员的网段。路由器可以跨网络、在网络之间以及跨 Internet 转发多播通信。
- 接收主机通知本地路由器要加入组，然后，接收发送到该组地址的所有后续数据报。
- 如果接收主机退出组，并且检测到该主机可能是子网上的最后一个组成员，则可以与本地路由器联系以退出组，同时通知路由器停止将多播数据报转发到该子网。

IP 多播的有如下优势：

（1）通过多播可以在网络上有效地支持高带宽、一对多的应用程序：

- 多播通信只需要发送数据的一个副本，便能够把报文传递给多台主机，从而显著地减少了网络通信量。
- 可以将主机配置为进行多播，而不必升级硬件。
- 由于较新的路由器已支持多播转发协议和多播路由协议，所以，在网络上启用多播是可行且经济的。

（2）多播适用于许多类型的一对多应用程序，例如：

- 多媒体。例如视频会议和协作计算。
- 网络资源的自动恢复。
- 数据传播。例如文件分发或数据库同步。
- 移动计算机支持。例如远程通讯簿更新。
- 组织出版物的分发。

4、多播的实现

IP 多播编程主要是通过使用 setsockopt()设置 IPPROTO_IP 层的选项来实现。涉及到的选项如**错误!未找到引用源。**所示：

表 1 多播编程相关的套接字选项

| 选项 | 选项值类型 | 说明 |
| --- | --- | --- |
| IP_ADD_MEMBERSHIP | ip_mreq | 加入到多播组 |
| IP_DROP_MEMBERSHIP | ip_mreq | 从多播组退出 |
| IP_MULTICAST_IF | unsigned long | 获取或设置打算从它上面发出多播数据的本地接口 |
| IP_MULTICAST_TTL | int | 为套接字获取或设置多播数据包的存活时间 |
| IP_MULTICAST_LOOP | bool | 如果 TRUE，发至多播地址的数据将原封不动地“反射”或“反弹”回套接字的进入缓冲区 |

加入和离开多播组的操作，分别用到 IPPROTO_IP 级的 IP_ADD_MEMBERSHIP（加入多播组）选项和 IP_DROP_MEMBERSHIP（离开多播组）套接字选项来实现。在使用这两个选项来加入或离开多播组时，必须传递一个 ip_mreq 结构来指出加入或离开的是哪一个多播组，其定义如下：

```
struct ip_mreq {
        struct in_addr imr_multiaddr;
        struct in_addr imr_interface;
}
```

其中，imr_multiaddr 用于指定要加入的多播组，imr_interface 指定发送多播数据的本地接口。如果将 imr_interface 设置为 INADDR_ANY，则选择使用默认网络接口，否则，使用指定的 IP 地址（有多个 IP 地址可选时）发送多播组。

例如，要加入多播组 234.5.5.5 的程序段如下所示：

```
struct ip_mreq ipmreq;
int len = sizeof(ipmreq);
ipmreq.imr_multiaddr.s_addr = inet_addr ("234.5.5.5");
ipmreq.imr_interface.s_addr = htonl (INADDR_ANY);
setsockopt (multiSock, IPPROTO_IP, IP_ADD_MEMBERSHIP, (char *) &ipmreq, len);
```

其中 multiSock 是一个已成功创建的数据报套接字。只要把上述程序段中的 IP_ADD_MEMBERSHIP 选项改为 IP_DROP_MEMBERSHIP，则可以从 ipmreq 中指定的多播组中退出。

选项 IP_MULTICAST_TTL 的作用是为套接字设置多播数据包的生存时间。当使用多址广播时，常常需要指定多播数据包的生存时间，该生存时间限制了多播报文的传播范围。生存时间由包括的路由网段来定义。生成时间为 0，指示多播消息不传送到“网线”上，但是可以在本地主机内部的多个套接字间传播。生存时间为 1 （默认值），指示将多播数据包传送到“网线”上，但是不跨越路由器。更高的取值决定了可以跨越的路由器数量。

选项 IP_MULTICAST_LOOP 的作用是：如果该选项为 TRUE，发至多播地址的数据将原封不动地“反射”或“反弹”回套接字的进入缓冲区。

多播程序的一般工作步骤为：

① 创建一个数据报套接字。比如：

```
int multiSock;
multiSock = socket (AF_INET, SOCK_DGRAM, 0);
```

② 把套接字与本地端口绑定，使得进程可以接收到多播数据。比如：

```
struct sockaddr_in local;
memset (&local, 0, sizeof (local));
local.sin_family = AF_INET;
local.sin_addr.s_addr = htonl(INADDR_ANY);//接收目标为本机地址的任意数据报
local.sin_port = htons (MULTIPORT);
bind (multiSock, (struct sockaddr*) &local, sizeof (local));
```

③ 用 setsockopt 设置 IP_ADD_MEMBERSHIP 选项，并指定要加入的多播组的地址结构。

若只是想向多播组发送数据，那只要以普通的 UDP 数据报向目的地址为多播地址来发送即可，不需要加入一个多播组；若想接收多播数据，则必须加入一个 IP 多播组。

## 五、实验步骤

根据实验要求，设计出程序主架构工作流程如**错误!未找到引用源。**所示：

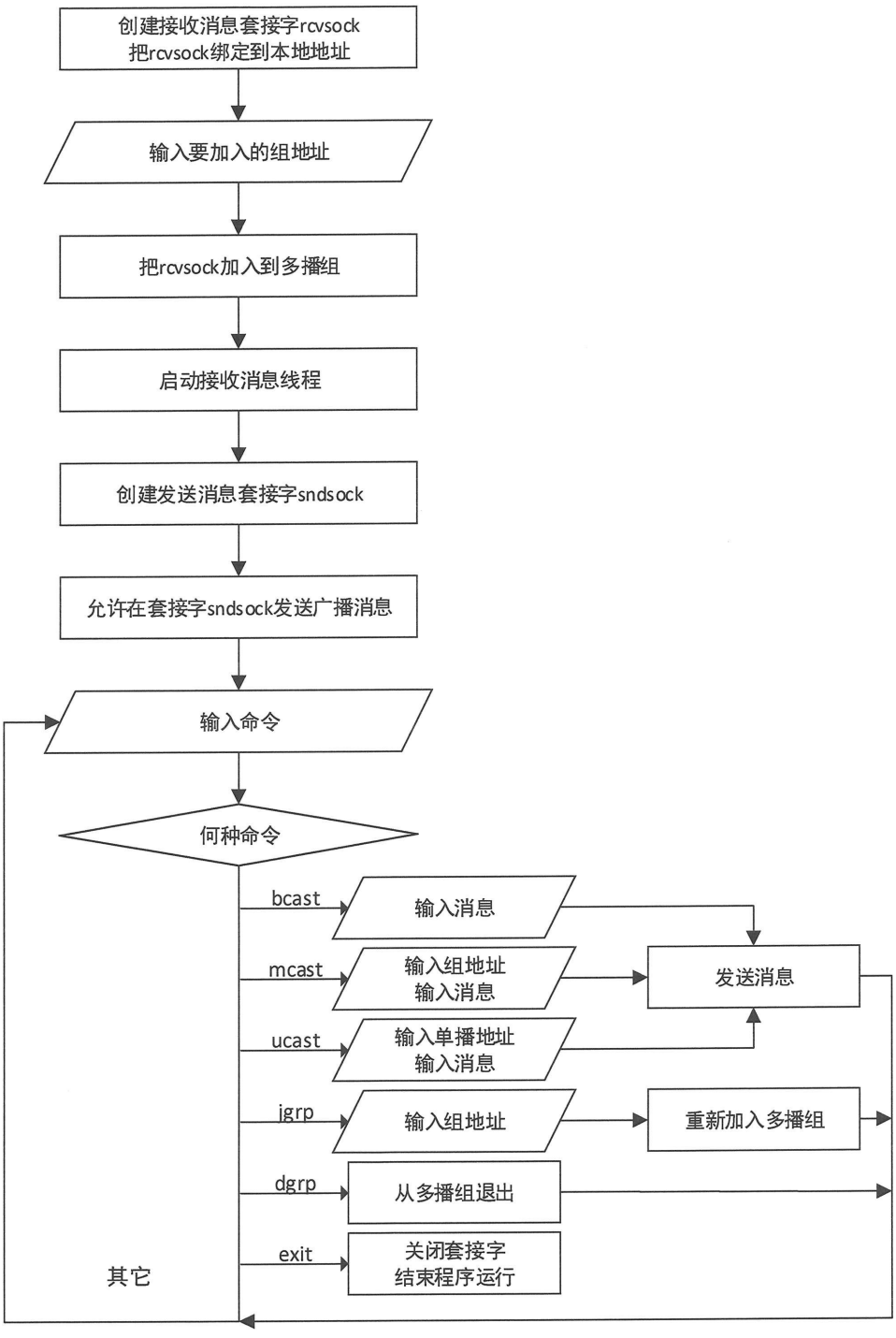



图 2-3 程序主流程图

1、声明部分全局变量如下：

```
#define BUFLEN 256
#define PORT 20000
```

```
#define IPADDRLEN 16
int sndsock = -1;                    //发送消息使用的套接字
int rcvsock = -1;                    //接收消息使用的套接字
pthread_t rmsgthread = -1;           //接收消息的线程标识符
struct sockaddr_in destip;           //消息接收方地址
socklen_t addrlen;
struct ip_mreq grpip;                //加入的多播组地址
```

2、消息接收函数

首先设置用于接收消息的套接字工作在非阻塞模式下，然后再在套接字上启动消息接收。若正确接收到消息，则把收到的消息以及发送方的 IP 地址和端口号显示出来。

```
void *rcvmsg (void* arg)
{
   int flags;
   int len;
   socklen_t alen;
   struct sockaddr_in raddr;
   char buff [BUFLEN];
     pthread_detach (pthread_self ());
   if ((flags = fcntl (rcvsock, F_GETFL, 0)) <0)
   {
         fprintf (stderr, "Error: %s\n", strerror (errno));
         exit (1);
   }
   flags |= O_NONBLOCK;
   if (fcntl (rcvsock, F_SETFL, flags) <0)
   {
         fprintf (stderr, "Error: %s\n", strerror (errno));
         exit (1);
   }
   while (1)
   {
         memset (buff, 0, sizeof (buff));
         alen = sizeof (raddr);
         if ((len = recvfrom(rcvsock, buff, sizeof(buff) -1, 0, (struct sockaddr *) &raddr, &alen)) > 0)
         {
               printf ("\nRecieve Msg: %s from: %s:%d\n",
                     buff, inet_ntoa (raddr.sin_addr), ntohs (raddr.sin_port));
         } else if (errno != EWOULDBLOCK)
         {
               rmsgthread = -1;
               return NULL;
         }
   }
```

```
}
```

3、广播消息发送函数

把程序设置的端口号和广播地址填充到结构变量 destip 中，然后再以 destip 为目的地址发送 UDP 数据报。函数参数 arg 指向了要发送的以'\0'结尾的字符串。

```
void broadcast (char *arg)
{
   addrlen = sizeof (struct sockaddr_in);
     memset (&destip, 0, addrlen);
     destip.sin_family = AF_INET;
     destip.sin_port = htons (PORT);
     destip.sin_addr.s_addr = htonl (INADDR_BROADCAST);
     sendto (sndsock, arg, strlen (arg), 0, (struct sockaddr *) &destip, addrlen);
}
```

4、多播报文发送函数

函数的参数是一个以'\0'结尾的字符串，该字符串由一个点分十进制形式的组播地址和待发送的消息组成，它们之间使用空格隔开。

程序在工作中首先把参数中的组播地址和消息分解开来，再利用分解得到的组播地址和程序设置的端口号来填充结构体变量 destip，最后以 destip 为目的地址把消息发送出去。

```
void multicast (char *arg)
{
   char gip [IPADDRLEN];
   char msg [BUFLEN];
   char *p;
   int len;
   int i;
   memset (gip, 0, IPADDRLEN);
   memset (msg, 0, BUFLEN);
   for (p = arg, i = 0; *p != ' ' && i < IPADDRLEN; p++, i++)
         gip [i] = *p;
   while (*p == ' ')
   {
         i++;
         p++;
   }
   if(*p)
   {
         len=strlen(arg)-i;
         memcpy (msg, p, len);
   }
   addrlen = sizeof (struct sockaddr_in);
```

```
    memset (&destip, 0, addrlen);
    destip.sin_family = AF_INET;
    destip.sin_port = htons (PORT);
    destip.sin_addr.s_addr = inet_addr (gip);
    sendto (sndsock, msg, strlen(msg), 0, (struct sockaddr *) &destip, addrlen);
}
```

5、单播报文发送函数

单播的发送与多播报文的发送相类似，只是使用的目的地址为单播地址而已。所以单播报文发送函数可以通过调用多播报文发送函数来直接实现单播报文的发送。代码如下：

```
void unicast (char *arg)
{
    multicast(arg);
}
```

6、加入多播组函数

加入多播组函数只有唯一的一个参数，即以点分十进制形式给出的组地址。在加入到多播组之前先关闭接收套接字和接收线程，从现有的多播组中退出，重新生成拟加入的多播组地址，然后再重新创建接收套接字、加入多播组、启动接收线程。

```
void joingroup (char *arg)
{
    int optlen;
    socklen_t addrlen;
    struct sockaddr_in localaddr;
    addrlen = sizeof (struct sockaddr_in);
    if (rmsgthread != -1)
            pthread_cancel (rmsgthread);
    if (rcvsock != -1)
            close (rcvsock);
    rcvsock = socket (AF_INET, SOCK_DGRAM, 0);
    memset (&localaddr, 0, addrlen);
    localaddr.sin_family = AF_INET;
    localaddr.sin_port = htons (PORT);
    localaddr.sin_addr.s_addr = htonl (INADDR_ANY);
    bind (rcvsock, (struct sockaddr*) &localaddr, addrlen);
      optlen = sizeof(struct ip_mreq);
    grpip.imr_multiaddr.s_addr = inet_addr(arg);
      grpip.imr_interface.s_addr = htonl (INADDR_ANY);
      setsockopt (rcvsock, IPPROTO_IP, IP_ADD_MEMBERSHIP, (char *) &grpip, optlen);
    pthread_create (&rmsgthread, NULL, rcvmsg, NULL);
    return;
```

```
}
```

7、从多播组中退出

```
void dropgroup(void)
{
   int optlen;
   optlen = sizeof (struct ip_mreq);
    setsockopt (rcvsock, IPPROTO_IP, IP_DROP_MEMBERSHIP, (char *) &grpip, optlen);
    return;
}
```

7、主函数

```
void main(void)
{
   char muip [IPADDRLEN];
   int code;
   char arg [BUFLEN];
   char *cmdline;
   size_t cmdlen;
   int len;
   addrlen = sizeof (struct sockaddr_in);
   int opt = 1;
   int optlen = sizeof (opt);
   sndsock = socket (AF_INET, SOCK_DGRAM, 0);
   setsockopt (sndsock, SOL_SOCKET, SO_BROADCAST, (char *) &opt, optlen);
   printf ("\nPlease multicast address:");
   scanf ("%s", muip);
   muip [IPADDRLEN-1] = '\0';
   joingroup (muip);
   while (1)
   {
        cmdlen = 0;
        while ((len = getline (&cmdline, &cmdlen, stdin)) < 0)
             fprintf (stderr, "Input command line error: %s\n", strerror(errno));
        cmdline[len-1] = '\0';
        memset (arg, 0, sizeof(arg));
        resolvecmd (cmdline, &code, arg);
        free (cmdline);
        switch (code)
        {
        case 0:
             broadcast (arg);
```

```
                break;
        case 1:
                multicast (arg);
                break;
        case 2:
                unicast (arg);
                break;
        case 3:
                joingroup (arg);
                break;
        case 4:
                dropgroup ();
                break;
        case 5:
                pthread_cancel (rmsgthread);
                close (sndsock);
                close (rcvsock);
                exit (0);
        default:
                break;
        }
    }
}
```

8、设计命令分析程序如下：

```
/*****************************************************************************
*               解析命令                                                    *
* 用法：resolvecmd (char *str, int *code, char *arg)                        *
* str:指向欲解析命令串的指针。命令串以'\0'结尾，命令与参数间使用空格隔开。  *
* 命令字不区分大小写。                                                      *
* code:指向保存返回命令代码的整型变量，代码为命令字在cmdlist数组中的序号。  *
* arg:用于返回命令参数                                                      *
* 本系统支持的命令保存在指针数组cmdlist中。                                 *
*****************************************************************************/
char *cmdlist[]={"BCAST","MCAST","UCAST","JGRP","DGRP","EXIT",NULL};
int resolvecmd(char *str,int *code,char *arg)
{
    int i;
    int len;
    char cmd[10];
    char *p;
    if(strlen(str)>=BUFLEN)
        return -1;
    memset(cmd,0,sizeof(cmd));
```

```
    for(i=0,p=str;i<sizeof(cmd) && (*p != ' ' && *p != '\0'); i++,p++)
        if( *p>='a' && *p <='z' )
            cmd[i] = *p - 'a' + 'A';
        else
            cmd[i] = *p;
    while(*p == ' ')
    {
        i++;
        p++;
    }
    if(*p)
    {
        len=strlen(str)-i;
        memcpy(arg,p,len);
    }
    *code=-1;
    for(i=0;cmdlist[i]; i++)
        if( !strcmp(cmd,cmdlist[i]))
        {
            *code = i;
            break;
        }
    return 1;
}
```

9、可能需要包含以下的头文件：

```
#include <stdlib.h>
#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <string.h>
#include <signal.h>
#include <pthread.h>
#include <fcntl.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
```